

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA
E.P DE INGENIERÍA DE SOFTWARE



Gestión de la configuración de software

Manual de Despliegue

DOCENTE:

Dra. Marlene Reyes

ALUMNOS:

Bohorquez Huaranga Kevin Rhamses

León Chacón Gerardo Raúl

Lopez Leonardo

Rojas Rojas Max

Sanchez Gotea Edu Joseph

Sevan Reyes David

Taco Zavala Miguel Angel

Vera Rodriguez Cristian Leonardo

Tabla de Contenidos

1. Requisitos Previos	3
1.1. Software requerido	3
1.2. Cuentas y accesos	3
2. Entorno de Desarrollo Local	4
2.1. Clonar el repositorio	4
2.2. Configurar el backend	4
2.3. Configurar el frontend	5
3. Configuración de la Base de Datos	6
3.1. Crear servicio MySQL en Railway	6
3.2. Variables de conexión	6
3.3. Inicializar el esquema	7
4. Despliegue del Backend en Railway	8
4.1. Conectar repositorio a Railway	8
4.2. Variables de entorno en Railway	9
4.3. Verificar el arranque	9
5. Despliegue del Frontend en Netlify	10
5.1. Conectar repositorio a Netlify	10
5.2. Configuración de build	10
5.3. Proxy y redirects	11
6. Verificación Post-Despliegue	12
6.1. Checklist de verificación	12
6.2. Solución de problemas frecuentes	13
7. Apéndice: Arquitectura de Despliegue	14

1. Requisitos Previos

Antes de iniciar el proceso de despliegue, es necesario contar con el software, las cuentas de servicio y los permisos de acceso descritos en esta seccion.

1.1 Software Requerido

Herramienta	Version minima	Proposito
Python	3.10 o superior	Ejecucion del backend FastAPI
pip	Incluido con Python 3.4+	Gestor de paquetes Python
Node.js	18 LTS o superior	Compilacion del frontend React/Vite
npm	9 o superior	Gestor de paquetes JavaScript
Git	2.30 o superior	Control de versiones y despliegue
Cliente MySQL	Opcional	Verificacion directa de la base de datos

1.2 Cuentas y Accesos Necesarios

- **Railway:** railway.app. Requerida para hospedar el backend y la base de datos MySQL.
- **Netlify:** netlify.com. Requerida para hospedar el frontend React.
- **Repositorio Git:** Acceso de lectura y escritura al repositorio Git del proyecto.
- **Permisos Railway:** Permisos de administrador en el proyecto de Railway.

2. Entorno de Desarrollo Local

Esta seccion describe el procedimiento para levantar el proyecto de forma completa en una maquina local, con el proposito de realizar pruebas antes del despliegue en produccion.

2.1 Clonar el Repositorio

1. Ejecutar el siguiente comando en la terminal:

```
git clone <URL_DEL_REPOSITORIO>
cd Veterinaria
```

2.2 Configurar el Backend

1. Ingresar al directorio del backend:

```
cd VeterinariaClinica_Backend
```

2. Crear y activar el entorno virtual de Python:

```
# Windows
```

```
python -m venv venv
```

```
venv\Scripts\activate
```

3. Instalar las dependencias del proyecto:

```
pip install -r requirements.txt
```

4. Crear el archivo de variables de entorno .env en la raíz de VeterinariaClinica_Backend con el siguiente contenido de referencia:

```
DATABASE_URL+pymysql=mysql://USUARIO:PASSWORD@HOST:3306/NOMBRE_BD
```

```
MYSQL_HOST=localhost
```

```
MYSQL_PORT=3306
```

```
MYSQL_USER=root
```

```
MYSQL_PASSWORD=su_password
```

```
MYSQL_DATABASE=veterinaria_db
```

```
SECRET_KEY=clave_secreta_larga
```

```
ALGORITHM=HS256
```

```
ACCESS_TOKEN_EXPIRE_MINUTES=1440
```

```
PROJECT_NAME=Sistema Veterinaria API
```

```
VERSION=1.0.0
```

```
DEBUG=True
```

```
ENVIRONMENT=development
```

```
BACKEND_CORS_ORIGINS=["http://localhost:5173"]
```

5. Iniciar el servidor de desarrollo:

uvicorn main:app --reload --host 0.0.0.0 --port 8000

6. Verificar el funcionamiento accediendo a las siguientes rutas desde el navegador:

`http://localhost:8000/health` # Estado del servidor

`http://localhost:8000/docs` # Documentacion Swagger UI

`http://localhost:8000/stats` # Estadisticas del sistema

Nota. El modo `--reload` reinicia el servidor automaticamente ante cualquier cambio en el codigo fuente.

2.3 Configurar el Frontend

1. Abrir una nueva terminal e ingresar al directorio del frontend:

`cd Veterinaria_BD`

2. Instalar las dependencias de Node.js:

`npm install`

3. Iniciar el servidor de desarrollo Vite:

`npm run dev`

4. Acceder a la aplicacion en el navegador:

`http://localhost:5173`

Nota. El archivo `vite.config.js` configura un proxy que redirige automaticamente todas las solicitudes `/api` al backend en `localhost:8000`. Ambos servicios deben estar en ejecucion simultaneamente.

3. Configuración de la Base de Datos

El sistema utiliza MySQL como motor de base de datos. En el entorno de producción, la instancia MySQL se hospeda en Railway dentro del mismo proyecto que el backend, comunicándose a través de la red interna.

3.1 Crear el Servicio MySQL en Railway

1. Iniciar sesión en railway.app.
2. Ingresar al proyecto existente o crear uno nuevo.
3. Hacer clic en el botón "+ New", seleccionar "Database" y luego "Add MySQL".
4. Esperar a que Railway aprovisiona el servicio. El proceso tarda aproximadamente un minuto.
5. Dirigirse a la pestaña "Variables" del servicio MySQL para obtener las credenciales de conexión.

Nota. Railway provee dos tipos de conexión: interna (mysql.railway.internal) para servicios dentro del mismo proyecto, y externa para conexiones desde fuera de Railway. Para el backend se utiliza la conexión interna.

3.2 Variables de Conexión

Las siguientes variables deben configurarse en el servicio del backend dentro del panel de Railway:

Variable	Valor de ejemplo	Descripción
MYSQL_HOST	mysql.railway.internal	Host interno de Railway
MYSQL_PORT	3306	Puerto estándar de MySQL
MYSQL_USER	root	Usuario generado por Railway
MYSQL_PASSWORD	xxxxxxxxxxxxxxxxxx	Contraseña generada por Railway

MYSQL_DATABASE	railway	Nombre de la base de datos
DATABASE_URL+pymysql	mysql://root:PASS@mysql.railway.internal/railway	URL completa de conexion

3.3 Inicializar el Esquema de la Base de Datos

El proyecto no utiliza migraciones automatizadas con Alembic. El esquema se gestiona de las siguientes formas:

- **Automatica:** SQLAlchemy ejecuta `create_all` al arrancar el backend, creando las tablas que no existen.
- **Manual:** Si se requiere recrear la base de datos, la estructura completa se infiere de los modelos ubicados en `app/models/`.

4. Despliegue del Backend en Railway

El backend FastAPI se despliega en Railway utilizando Gunicorn con workers Uvicorn. Railway detecta automaticamente el archivo `Procfile` del proyecto y ejecuta el comando de arranque.

4.1 Conectar el Repositorio a Railway

1. En el proyecto Railway, hacer clic en "+ New" y seleccionar "GitHub Repo".
2. Autorizar Railway para acceder a la cuenta de GitHub y seleccionar el repositorio del proyecto.
3. En la configuracion del servicio, ir a "Settings" > "Source" y establecer el directorio raiz en:

`VeterinariaClinica_Backend`

4. Railway detecta el archivo `Procfile` automaticamente. Dicho archivo contiene:

```
web: gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:$PORT
```

Nota. Railway inyecta automaticamente la variable de entorno `PORT`. No debe definirse de forma manual.

4.2 Variables de Entorno en Railway

En la pestana "Variables" del servicio backend, agregar las siguientes variables:

Variable	Valor de ejemplo	Descripción
DATABASE_URL+pymysql	mysql://root:PASS@mysql.railway.internal/railway	URL de conexión a MySQL
MYSQL_HOST	mysql.railway.internal	Host interno Railway
MYSQL_PORT	3306	Puerto MySQL
MYSQL_USER	root	Usuario de la base de datos
MYSQL_PASSWORD	su_password_railway	Contraseña de la base de datos
MYSQL_DATABASE	railway	Nombre de la base de datos
SECRET_KEY	clave_secreta_robusta	Clave de firmado (mínimo 32 caracteres)
ALGORITHM	HS256	Algoritmo de firma
ACCESS_TOKEN_EXPIRE_MINUTES	1440	Expiración de tokens en minutos
PROJECT_NAME	Sistema Veterinaria API	Nombre del proyecto
VERSION	1.0.0	Version del sistema
DEBUG	False	Modo depuración (False en producción)
ENVIRONMENT	production	Entorno de ejecución
BACKEND_CORS_ORIGINS	["https://su-app.netlify.app"]	URLs permitidas para CORS

Nota. Railway permite referenciar variables de otro servicio del mismo proyecto mediante la opcion "Reference Variable", evitando la copia manual de las credenciales de MySQL.

4.3 Verificar el Arranque

1. Revisar los registros de despliegue en: servicio > "Deployments" > ultimo deploy > "View Logs".
2. Confirmar que el servidor arranco correctamente. Los registros deben mostrar:

[INFO] Starting gunicorn 21.2.0

[INFO] Listening at: http://0.0.0.0:PORT

[INFO] Booting worker with pid: ...

Application startup complete.
3. Habilitar el dominio publico en "Settings" > "Networking" > "Public Networking" y copiar la URL generada.
4. Probar el endpoint de verificacion de estado:

GET https://su-backend.up.railway.app/health

Respuesta esperada: { "status": "ok" }

5. Despliegue del Frontend en Netlify

El frontend React/Vite se despliega en Netlify. El archivo netlify.toml incluido en el repositorio configura automaticamente el proceso de construccion, el proxy hacia el backend y el manejo de la aplicacion de pagina unica (SPA).

5.1 Conectar el Repositorio a Netlify

1. Iniciar sesion en netlify.com con la cuenta de GitHub.
2. Hacer clic en "Add new site" > "Import an existing project" > "GitHub".
3. Autorizar Netlify y seleccionar el repositorio del proyecto.

5.2 Configuración de Build

Configurar los siguientes campos en el asistente de Netlify. Estos valores ya están definidos en `netlify.toml`:

Campo	Valor
Base directory	Veterinaria BD
Build command	npm run build
Publish directory	dist
Node version	18 o superior

5.3 Proxy y Redirects

El archivo `netlify.toml` gestiona automáticamente los siguientes comportamientos:

- **Proxy de API:** Todas las solicitudes a `/api/*` se redirigen al backend en Railway, resolviendo restricciones de CORS en producción.
- **SPA fallback:** Cualquier ruta no encontrada se redirige a `/index.html`, habilitando la navegación gestionada por React Router.
- **Cabeceras de seguridad:** Incluye X-Frame-Options, X-XSS-Protection, X-Content-Type-Options y Referrer-Policy.

Nota. Si la URL del backend cambia, se debe actualizar la línea 9 del archivo `netlify.toml` con el nuevo valor y realizar un commit para que Netlify redesplice el sitio automáticamente.

1. Hacer clic en "Deploy site" para iniciar el proceso de construcción y publicación.
2. Netlify asignará una URL automáticamente. Es posible configurar un dominio personalizado desde las opciones del sitio.

6. Verificacion Post-Despliegue

Una vez completado el despliegue de los tres componentes, se deben realizar las siguientes verificaciones para confirmar el correcto funcionamiento del sistema.

6.1 Checklist de Verificacion

Backend (Railway)

- GET /health responde con estado 200 y el cuerpo { "status": "ok" }.
- GET /stats retorna datos estadisticos del sistema.
- GET /docs carga la interfaz Swagger UI correctamente.
- Los registros de Railway no presentan errores de conexion a la base de datos.

Base de Datos

- El servicio MySQL en Railway muestra estado activo.
- El backend se conecta sin errores (verificable en los registros).
- Las tablas se crean automaticamente en el primer arranque.

Frontend (Netlify)

- La URL de Netlify carga la pantalla de seleccion de rol.
- El inicio de sesion funciona con los tres roles: Administrador, Veterinario y Recepcionista.
- Las solicitudes a /api/* se redirigen correctamente al backend en Railway.
- No se producen errores 404 al navegar entre las secciones del sistema.

Flujo de Extremo a Extremo

- Crear un cliente desde el rol Recepcionista.
- Registrar una mascota asociada al cliente.
- Crear una solicitud de atencion.

- Agendar una cita.
- Realizar el triaje desde el rol Veterinario.
- Completar una consulta y agregar un diagnostico.

6.2 Solucion de Problemas Frecuentes

Error 502 Bad Gateway en Railway

Causa probable: *El servidor no arranco correctamente.*

- Revisar los registros en Railway > Deployments > View Logs.
- Verificar que el Procfile usa \$PORT: --bind 0.0.0.0:\$PORT.
- Confirmar que requirements.txt incluye gunicorn y uvicorn[standard].
- No definir la variable PORT de forma manual; Railway la inyecta automaticamente.

Error de CORS en el navegador

Causa probable: *El frontend no puede comunicarse con el backend.*

- Verificar que BACKEND_CORS_ORIGINS incluye la URL de Netlify.
- Confirmar que el redirect /api/* en netlify.toml apunta a la URL correcta del backend.
- Reconstruir el sitio en Netlify tras modificar netlify.toml.

Pantalla en blanco en Netlify

Causa probable: *React Router no puede resolver la ruta solicitada.*

- Verificar que netlify.toml contiene el redirect SPA: /* hacia /index.html con estado 200.
- Confirmar que el directorio de publicacion es dist y el comando de build es npm run build.

Error de conexion a la base de datos

Causa probable: *El backend no puede establecer conexion con MySQL.*

- Verificar que MYSQL_HOST es mysql.railway.internal (conexion interna, no externa).

- Confirmar que el servicio MySQL y el backend pertenecen al mismo proyecto de Railway.
- Revisar que MYSQL_PASSWORD y MYSQL_USER coinciden con los generados por Railway.

Inicio de sesion falla con credenciales validas

Causa probable: *El sistema rechaza la autenticacion del usuario.*

- Verificar que el usuario existe en la base de datos.
- Tener en cuenta que las contraseñas se almacenan actualmente en texto plano.
- Revisar los registros del backend para obtener el error exacto del endpoint /auth/login.

7. Apendice: Arquitectura de Despliegue

7.1 Archivos de Configuracion Clave

Archivo	Proposito
VeterinariaClinica Backend/Procfile	Comando de arranque en Railway
VeterinariaClinica Backend/requirements.txt	Dependencias Python del backend
VeterinariaClinica_Backend/.env	Variables de entorno locales (no versionar en produccion)
VeterinariaClinica_Backend/app/config/database.py	Configuracion del motor SQLAlchemy
Veterinaria_BD/netlify.toml	Configuracion de build, redirects y cabeceras en Netlify
Veterinaria BD/vite.config.js	Proxy de desarrollo y configuracion de build Vite

7.2 URLs del Sistema en Produccion

Componente	URL	Descripcion
Frontend	https://su-app.netlify.app	Aplicacion web principal
Backend	https://veterinariaclinicabackend-production.up.railway.app	API REST
Docs API	https://veterinariaclinicabackend-production.up.railway.app/docs	Swagger UI
Health	https://veterinariaclinicabackend-production.up.railway.app/health	Estado del servidor

7.3 Consideraciones de Seguridad

El sistema presenta las siguientes vulnerabilidades conocidas que deben resolverse antes de un lanzamiento en produccion formal:

- **Contrasenas:** Las contraseñas de usuario se almacenan en texto plano. Se recomienda implementar `passlib[bcrypt]` para el hash de contraseñas.
- **Autenticacion:** El modulo `python-jose` esta instalado pero no emite ni valida tokens JWT. La sesion persiste en `localStorage` del navegador.
- **CORS:** El valor `allow_origins=["*"]` en `main.py` permite solicitudes desde cualquier origen. Debe restringirse a las URLs del frontend.
- **Credenciales:** El archivo `.env` con credenciales esta actualmente versionado en el repositorio. Debe agregarse a `.gitignore` y rotarse las credenciales.